



SlimTorq Simulator — User

Manual

FOC · PWM · INVERTER · LCL · ENCODER

SlimTorq is a closed-loop simulator for a slotless PMSM driven by Field-Oriented Control. The plant is an OpenModelica FMU (`SlotlessPMSM_abc.fmu`); the FOC controller, encoder, PWM inverter, and optional LCL filter run in Python and update the FMU at the inner sim step. Configure a run from the left panel, click **Simulate**, read the plots on the right. Every Simulate click runs a fresh sim and overwrites `.temp/<family>_<variant>.parquet`.

INSTALL & RUN (DOCKER, ONE CALL)

Only requirement on the host is Docker Engine or Docker Desktop (docs.docker.com/get-docker). The published image bundles the Python deps, motor catalog, assets, and the pre-built FMU — no Python or OpenModelica toolchain needed.

1. Install Docker on your PC (Linux, macOS, or Windows + WSL2).
2. From the repo root run: `docker compose up`
3. Open `http://localhost:8000` in your browser.

Compose pulls (or builds) the image, mounts a named volume for the parquet artifacts, and starts the service. All runtime settings — port, CORS, frontend bundle path, artifact directory, catalog path — live in `config/config.yaml` and can be overridden per-env with the double-underscore convention (e.g. `APP_PORT=9000`, `ARTIFACTS_DIR=/var/data`).

CONFIG PANEL — WHAT EACH FIELD DOES

Motor

variant SlimTorq catalog variant. R_s , L_s , λ_{PM} , p , J , and the BEMF cap are derived from this. V_{dc} is auto-filled to $1.5 \times \text{rated_voltage}$ when the variant changes; override via the V_{dc} field below.

Power stage

V_{dc} DC bus voltage (V). Defaults to $1.5 \times \text{motor.rated_voltage}$ — the industrial-servo rule of thumb that leaves the PI controller headroom for the steady-state $R \cdot i_q$ drop and the transient $L \cdot di_q/dt$. Lower it (e.g. $= \text{rated_voltage}$) to demonstrate undersized-bus saturation; raise it for more headroom at the cost of larger PWM ripple ($\Delta I_{pp} \propto V_{dc}$).

f_{pwm} PWM carrier frequency (Hz). Sets the inner sim step ($T_{pwm}/20$ by default) and the current-loop bandwidth $\approx f_{pwm}/9.4$.

t_{dead} Gate-driver dead time (s). 0 disables. Adds 5th/7th harmonic distortion to i_{abc} .

PWM mode `sine`, `svpwm`, `dpwmmax`, `dpwmmin`, `dpwm1`, or `auto` (`svpwm` $\leftrightarrow$ `dpwm1` at $m_i = 0.45/0.55$). DPWMx clamps one phase to a rail, ~33% less switching loss.

Output filter (LCL)

enable Insert LCL low-pass between inverter terminals and motor; engages active-damping observer + K_d injection.

f_c Target cutoff. L_f , C_f , R_d derived as $L_f=L_s$ (matched), $C_f=1/(\omega_c^2 \cdot L_f)$, $R_d=\sqrt{(L_f/C_f)}/3$. $f_{res} \approx \sqrt{2} \cdot f_c$.

ζ_{target} Active-damping target damping ratio of the dominant LCL pole pair. Drives $K_d = 2 \cdot \zeta \cdot \sqrt{(L_{total}/C_f)} - (R_1 + R_d)$. 0.7 (critical) default; lower $\rightarrow$ faster but oscillatory; higher $\rightarrow$ over-damped.

α_{obs} Observer pole multiplier ($\times \omega_{res}$). Places the Luenberger observer's three eigenvalues coincident-

Trajectory

t_{end}, t_{step} Sim horizon and load-step time (s).

T_L^{ref}/T_e^{peak} Step amplitude as fraction of catalog peak torque.

T_f Run-length override; *None* = auto.

Timing

Δt_{sim} Inner sim step (s); *None* = $T_{pwm}/20$. Must $\leq T_{pwm}/10$ (enforced). Use $\leq T_{pwm}/50$ for clean switching-mode results.

Current loop — PI tuning

Each rule is *plant-aware*. With the LCL filter **off** the plant is the LR motor (R_s, L_s); the user's `pi_mode` selects the rule.

With the LCL filter **on**, the drivetrain auto-selects Skogestad on the low-frequency equivalent $L_{eq}=L_1+L_{load}$, $R_{eq}=R_1+R_{load}$ (rejecting bandwidths within $\omega_{res}/5$ of resonance — the active-damping margin), and additionally computes an analytic active-damping gain $K_d = 2 \cdot \zeta \cdot \sqrt{(L_{total}/C_f)} - (R_1 + R_d)$.

FOCController subtracts $-K_d \cdot \hat{i}_{c_hat}$ from v_{dq}^{ref} before vector saturation, with $\hat{i}_{c_hat}$ from the LCL observer (zero-cycle delay).

Modulus Optimum $K_p=L \cdot f_{pwm}/3$, $K_i=R \cdot f_{pwm}/3$. Default. $\approx 63^\circ$ PM, 4.3 % overshoot. (LR plant only.)

Skogestad SIMC $K_p=L/(T_c+\tau)$, $T_i=\min(L/R, k_1 \cdot (T_c+\tau))$. $T_c=None$ uses $\tau=1.5/f_{pwm}$; $k_1=1.44$ default $\rightarrow \sim 3\times$ sharper load-step rejection. Auto-applied when LCL is on.

ζ_{target} Damping ratio of the dominant LCL-plant pole pair under active damping. 0.7 (critically damped) default; lower for faster but oscillatory; higher for over-damped. Drives K_d .

Manual Type K_p/K_i directly. Bypasses the LCL safety gate.

real at $-\alpha \cdot \omega_{\text{res}}$ via Ackermann. Higher $\alpha \rightarrow$ faster estimate, more noise amplification. 3.0 default.

Encoder (inductive flux model)

N_{bits}	Absolute resolution; $\text{LSB} = 2\pi/2^N$.
θ_{offset}	Constant mounting offset (rad).
A_k, k_k, φ_k	Three cyclic harmonics: $\theta_{\text{err}} = \sum A_k \cdot \sin(k_k \cdot \theta + \varphi_k)$.
$T_{s,\text{enc}}$	Encoder sample period (s). ZOH between samples. Visible as sawtooth $\omega \cdot T_{s,\text{enc}}$ on the encoder-error plot.



Reading the plots

12 PANELS · WHAT TO LOOK FOR

The plot column renders 12 Plotly figures from the most-recent parquet. Each figure pairs a measurement (solid) with its reference (dashed) using the same palette slot per signal. Steady-state metrics are computed on the trailing 80 % of samples to skip the load-step transient.

Tracking

Torque (T_L^{ref} vs T_e), i_q^* vs i_q , i_d^* vs i_d . Each subplot title carries the steady-state RMS reference-tracking error in %.

PI performance

e_d , e_q errors; v_d^{ref} , v_q^{ref} ; $|v_{dq}|$ vs the $V_{\text{max}} = V_{\text{dc}}/2$ vector limit (red-dotted). PI-saturation rug strip below shows when the integrator was frozen.

dq round-trip

$v_d^{\text{actual}} - v_d^{\text{ref}}$, $v_q^{\text{actual}} - v_q^{\text{ref}}$, reconstructed from logged post-inverter v_{abc} via Clarke+Park. **Cycle-average ≈ 0** proves inverter scaling is correct. Switching-mode shows ripple; average/ideal modes show flat zero.

i_q zoom

$5 \times T_{\text{pwm}}$ window in steady state with carrier-valley guides. Shows the controller's actual measurement noise vs reference.

i_q FFT, ω FFT, i_a FFT

Log-magnitude spectra (trailing 80 %, DC removed). Switching ripple shows at $2 \cdot f_{\text{pwm}}$ in i_q , at $f_{\text{pwm}} \pm n \cdot f_1$ in i_a . Dead-time signature: $5^{\text{th}}/7^{\text{th}}$ of the electrical fundamental in i_a .

Debug section

Inverter mode **ideal** = $v_{\text{abc}}^{\text{ref}}$ straight to FMU; **average** = duty calc but cycle-averaged voltage (no dead-time, isolates scaling bugs); **switching** = real PWM compare + dead-time (default).

Sticky controls. Config panel stays pinned while the plot column scrolls. The **Simulate** button sticks to the bottom of the config panel so it's always reachable. Hard-refresh after CSS edits if the page looks stale.

Phase currents (abc)

i_a , i_b , i_c overlaid. With switching enabled and no filter, ripple amplitude $\Delta i_{\text{pp}} \approx V_{\text{dc}} \cdot T_{\text{pwm}} / (8 \cdot L_s)$.

Phase voltages

v_a^{ref} dashed, post-inverter v_a faint solid, and (when LCL on) v_a^{motor} at filter output as medium solid. With filter on, the motor trace should hug the smooth ref while the inverter trace hits $\pm V_{\text{dc}}/2$.

PWM duties

d_a , d_b , d_c . With **svpwm** / **dpwmX** the shape clearly departs from pure sine — visual confirmation the zero-sequence injection took effect.

Encoder error

$\text{wrap}(\theta_m^{\text{true}} - \theta_m^{\text{meas}})$ in mrad. Dashed envelope at $\omega \cdot T_{s,\text{enc}}$ is the ZOH bound; the trace should ride inside this band. Excursions outside it are cyclic harmonics + quantization + offset.

Speed + saturation

ω_m^{true} vs ω_m^{meas} , plus a rug strip marking d/q PI saturation events.

Observer (LCL only)

Appears next to *Signal Processing* when the LCL filter is enabled. Two panels overlay the measured dq currents (i_d^{meas} , i_q^{meas}) with the Luenberger observer's three estimated LCL states (i_1 , v_c , i_m) and the derived capacitor current $i_c = i_1 - i_m$. The observer's gain L is pole-placed at $\alpha_{\text{obs}} \cdot \omega_{\text{res}}$ via Ackermann. **Active damping is closed:** FOCController consumes i_c^{hat} as $v_{\text{damp}} = -K_d \cdot i_c^{\text{hat}}$ per axis (logged as **ad_d**, **ad_q** in the parquet), with K_d set analytically by Z_{target} . The FFT view shows residual spectral content after damping.

LOG OUTPUT — WHAT'S IN THE PARQUET

Each Simulate POST writes `<artifacts.dir>/<family> <variant>.parquet` — a full per-sample log of the inner-loop signals plus a key-value metadata block. The same file is what the UI fetches as Apache Arrow IPC to render the 12 plots; external tools (polars, duckdb, pandas) read it unchanged.

Per-sample columns (rows = inner ticks)

Time	<code>t</code>
References	<code>TL_ref</code> , <code>i_d_ref</code> , <code>i_q_ref</code> , <code>v_d_ref</code> , <code>v_q_ref</code> , <code>v_a_ref</code> , <code>v_b_ref</code> , <code>v_c_ref</code>
Measurements	<code>T_e</code> , <code>i_d_meas</code> , <code>i_q_meas</code> , <code>i_a</code> , <code>i_b</code> , <code>i_c</code> , <code>θ_m_true</code> , <code>θ_m_meas</code> , <code>ω_m_true</code> , <code>ω_m_meas</code>
PWM duties	<code>d_a</code> , <code>d_b</code> , <code>d_c</code>
Switching states	<code>s_a</code> , <code>s_b</code> , <code>s_c</code> (int8)
Post-inverter	<code>v_a</code> , <code>v_b</code> , <code>v_c</code>
Post-LCL motor terminals	<code>v_a_motor</code> , <code>v_b_motor</code> , <code>v_c_motor</code> (= post-inverter when no LCL)
Modulation depth	<code>m_index</code> , <code>pwm_mode_active</code> (int8 code, mode-active-this-tick)
LCL observer (NaN when filter off)	<code>il_d_hat</code> , <code>vc_d_hat</code> , <code>im_d_hat</code> , <code>ic_d_hat</code> , <code>il_q_hat</code> , <code>vc_q_hat</code> , <code>im_q_hat</code> , <code>ic_q_hat</code>
Active damping	<code>ad_d</code> , <code>ad_q</code> (= $-K_d \cdot i_{c_hat}$ per axis; 0 when no LCL)
Saturation flags	<code>sat_d</code> , <code>sat_q</code> (bool)

Parquet key/value metadata (`slimtorq.*`)

Identity	<code>schema_version</code> , <code>params_hash</code> (blake2b-8 of canonical params), <code>params_json</code> (full SimParams)
FOC tuning	<code>foc_kp</code> , <code>foc_ki</code> , <code>pi_mode</code> , <code>pi_tc</code> , <code>pi_kl</code>
Power stage	<code>vdc</code> (resolved bus voltage), <code>f_pwm</code> , <code>t_dead</code> , <code>ts_enc</code> , <code>inverter_mode</code> , <code>pwm_mode</code>
LCL filter	<code>filter_enabled</code> , <code>filter_fc</code> , <code>filter_lf</code> , <code>filter_cf</code> , <code>filter_rd</code>
Mechanical	<code>b_est</code> (steady-state viscous-friction estimate from trailing 20%)
Motor	<code>motor_family</code> , <code>motor_name</code> , <code>motor Rated voltage</code> , <code>r_s</code> , <code>l_s</code> , <code>pole_pairs</code> , <code>i_q_peak</code> , <code>te_peak_ls</code>

SimMeta (returned by `POST /api/simulate`)

Run identity	<code>params_hash</code> , <code>artifact_name</code> , <code>rows</code>
Tracking	<code>err_pct</code> (RMS $i_q^{meas} - i_q^{ref}$, trailing 80%, % of $i_{q,peak}$)
Resolved gains	<code>foc_kp</code> , <code>foc_ki</code> , <code>pi_mode</code>
Motor	<code>motor_family</code> , <code>motor_name</code> , <code>rated_voltage</code> (= V_{dc} used)
Ripple — i_q	<code>iq_ripple</code> = { <code>delta_pp</code> , <code>pct_rated</code> , <code>pct_cmd</code> } (controller dq frame; full-tail pp)
Ripple — T_e	<code>te_ripple</code> = { <code>delta_pp</code> , <code>pct_rated</code> , <code>pct_cmd</code> } (motor torque; full-tail pp)
Ripple — i_a	<code>ia_ripple</code> = { <code>delta_pp</code> , <code>pct_rated</code> , <code>pct_cmd=null</code> } (phase current; windowed pp $\sim 2 \cdot T_{pwm}$ to reject fundamental)
Parquet metadata	<code>parquet_meta</code> : dict echo of the slimtorq.* keys above

Reading external. Polars: `pl.read_parquet(path)` for columns;
`pyarrow.parquet.read_metadata(path).metadata` for the slimtorq.* keys. The trace is over-written on every Simulate; if you need to keep a run, copy it out of `.temp/` before the next click.

WORKFLOW TIPS

- **Diagnose noise:** bypass with `Inverter mode = ideal`. If clean, the inverter is the source. Then try `average` — non-zero `mean  $\Delta v_d$` in the dq round-trip means a scaling bug; flat zero means switching ripple is the issue.
- **Tune for disturbance rejection:** switch PI to Skogestad, lower k_i toward 1.44. Tighter integrator → smaller `err %` in the tracking subplot titles.
- **Pick f_c for the LCL:** aim near $f_{pwm}/10$ for ~40 dB carrier attenuation. The hint under the input gives the loop-safe floor for the current f_{pwm} .
- **Read the ripple percentages carefully:** the Header shows $\Delta i_{q,pp}$, $\Delta T_{e,pp}$, $\Delta i_{a,pp}$ as % of *continuous* rating. When the motor runs above continuous (typical for short load-step peaks), these naturally exceed 100% even with good tracking — cross-reference the absolute Δ values and per-command percentages. $\Delta i_{a,pp}$ is computed over short sliding windows (~2 PWM periods) to isolate PWM ripple from the rotating fundamental.
- **Voltage / ripple trade-off:** raising V_{dc} gives the PI more headroom (helps high-current low-V windings escape saturation) but raises PWM current ripple linearly. The four levers (V_{dc} , f_{pwm} , LCL on/off, Z_{target}) are co-designed — turning one usually requires tuning at least one other.
- **Sample-rate aliasing check:** at fixed f_{pwm} , halve Δt_{sim} . If noise drops, it's edge quantization in the modulator, not real inverter physics.
- **Output:** each run writes `.temp/<family>_<variant>.parquet` with full per-sample log and `slimtorq.*` metadata (params hash, tuning, modes, filter values). `SimMeta` JSON returned by `POST /api/simulate` additionally carries iq/te/ia ripple stats.